VocalNote

Offline Local Document-to-Podcast Generation System

Complete Project & Architecture Report

An offline, privacy-first AI application designed to transform PDF documents into highly natural-sounding, two-speaker podcast discussions. Built utilizing FastAPI, llama-cpp-python, Kokoro-82M TTS, and a React + TypeScript frontend, the system runs entirely locally on consumer hardware without internet dependencies or cloud APIs.

Author/Developer: vocalnote-main Contributors

Target Platforms: Windows, macOS (Apple Silicon Optimized), Linux

Report Generated: June 2026

1. Executive Summary & Overview

VocalNote addresses the critical need for secure, offline, and high-performance document synthesis. Modern cloud solutions (such as NotebookLM) present substantial privacy risks when dealing with sensitive academic, medical, or corporate files. VocalNote solves this by constructing a local pipeline that auto-detects hardware configurations, accelerates inference using MPS (Metal Performance Shaders) on Apple Silicon, and coordinates local GGUF Large Language Models alongside the state-of-the-art Kokoro Text-To-Speech engine.

The project is structured as a decoupled web application with a FastAPI Python backend serving endpoints to a React and TypeScript frontend built on Vite. For distribution, it leverages advanced Docker containerization (multi-stage builds) allowing the entire frontend-backend application to run in a single monolith image.

Core Architecture Pillars

- **Offline Security:** Zero data egress. Models run locally inside memory. No API keys required.
- **Hardware Autodetect:** Configures optimal thread count and offloads LLM layers to Apple Silicon GPU automatically.
- **Pipeline v2 Speeds:** Combines parsing, in-memory caching, and NumPy audio array concatenation to reduce latency.
- **Monolithic Docker:** Serves the frontend bundle directly from FastAPI static mount, running on a single container port.

2. Tech Stack breakdown

Backend Stack Components

- Python 3.11: Core execution language.
- FastAPI: High-performance, async framework to route requests and stream wav files.
- llama-cpp-python: Python bindings for llama.cpp, enabling local GGUF inference with optimized thread counts.
- Kokoro (Kokoro-82M): An ultra-compact text-to-speech model. Highly expressive, realistic, and fast.
- PyPDF2: Extractor library to pull raw strings from incoming PDF payloads.
- LangChain Text Splitters: Used to parse raw text into clean overlap-friendly paragraphs (3000 chars) for the LLM.
- SciPy, NumPy, Pydub: High-performance audio manipulation and format conversion.
- Uvicorn: ASGI web server running on port 9000.

Frontend Stack Components

- React 18: Client UI framework.
- TypeScript 5: Type safety across React components.
- Vite 5: Modern frontend builder set to port 2000 for development.
- Axios: Promise-based HTTP client to submit multipart/form-data PDF files and receive audio Blobs.
- Vanilla CSS: Custom-designed styling engine using traditional Indian themes (Mandala elements, Diya-style timers, Terracotta/Saffron variables).

3. Backend Code Breakdown

File: backend/podcast_workflow.py (Core Pipeline Engine)

The PodcastPipeline class manages the entire processing chain:

1. Initialization & Model Check: Downloads models if missing:
   - Qwen3-1.7B-Q8_0.gguf (Text Preprocessing & Cleaning Model)
   - qwen2.5-3b-instruct-q4_k_m.gguf (Podcast Script Dialog Generator)

2. Hardware Detection (detect_device): Detects platform (macOS Darwin vs. Windows/Linux).
   - On macOS with ARM, it configures device='mps' and n_gpu_layers=-1 (full GPU acceleration).
   - On Windows/Linux, it configures device='cpu' and optimizes n_threads to half of total system cores.

3. Text Extraction (extract_text_from_pdf): Reads up to 100k characters from uploaded PDFs using PyPDF2.

4. Preprocessing (preprocess_text): Splices PDF text into 3,000-character chunks with LangChain. Passes each chunk through the Qwen 1.7B model with a system prompt that strips out LaTeX, formatting glitches, and academic citations. This guarantees clean input text.

5. Transcript Generation (generate_transcript): The cleaned text is compiled and passed to Qwen 3B. A highly tailored system prompt forces the model to generate a podcast dialogue representing a conversation between Speaker 1 (host/educator) and Speaker 2 (curious student). Crucially, the system prompt forces the output to be formatted strictly as a Python list of tuples: [('Speaker 1', 'dialogue'), ('Speaker 2', 'dialogue')].

6. In-Memory TTS Synthesis (generate_podcast): Loads the Kokoro pipeline. Iterates over the dialog list and generates high-quality audio segments using 'am_fenrir' for Speaker 1 and 'bf_emma' for Speaker 2. Rather than writing chunks to disk sequentially, they are concatenated in-memory as a single NumPy array, then saved in one pass to podcast.wav.

4. FastAPI Endpoints & Frontend App

File: backend/api.py (API Routes)

- POST /upload-pdf: Saves a raw PDF file into the pipeline's temporary workspace.
- POST /generate-podcast: Performs PDF upload (if attached) and invokes the run_pipeline method in a separate asynchronous thread (using asyncio.to_thread). Once completed, it streams the WAV file back using FastAPI's FileResponse.
- GET /download-podcast: Fetches the cached podcast.wav from the server.
- Static Mount: Serves built React files in production mode: app.mount('/assets', StaticFiles(directory='dist/assets'))

File: frontend/src/App.tsx (React Controller)

The frontend operates as a single page application with modular components:

- State management: Tracks the selected file, current generation status, API errors, and the generated audio URL.
- File upload validation: Limits the file input directly to 'application/pdf'.
- Axios integration: Submits the PDF using FormData. It sets responseType: 'blob' to handle binary file streams. It also checks if the response header content-type is 'application/json' to handle errors gracefully.
- Blob URL instantiation: Translates the response data stream using window.URL.createObjectURL(new Blob(...)) to feed directly into a custom visual audio player.

Key Sub-components (frontend/src/components):
- Header.tsx: Renders a glassmorphic top navigation bar.
- HeroUpload.tsx: Renders the drag-and-drop workspace for PDF submission.
- StatusBar.tsx: Contains a stylized progress indicator (inspired by a glowing diya lamp).
- AudioPlayer.tsx: A fully custom visual waveform component that supports seeking and downloading generated audios.
- FeatureCards.tsx: A grid summarizing the pipeline steps.

5. Docker & System-Level Learnings

During the setup and engineering phases, several critical problems were solved:

1. Handling Large Files in Git vs Docker:
Git restricts files above 100MB, prohibiting raw GGUF files in code repositories. To bridge this in Docker, the project uses volume mounts mapping a folder on the host (~/Downloads/VocalNote_models) to /app/models inside the container. This ensures that the downloaded 4GB model files are preserved across container runs.

2. Docker macOS GPU Limitation:
Docker runs on macOS inside a Linux Virtual Machine. Apple Silicon's GPU (MPS/Metal Performance Shaders) is unavailable to virtualized guest OSes. Hence, Docker runs are strictly CPU-bound. Users seeking maximum speed (Metal GPU) are guided to execute Option B (Traditional Local Setup) directly on the host machine.

3. Multi-Stage Monolith Builds:
To avoid running two separate server containers (Vite node dev server and FastAPI), we use a multi-stage Dockerfile. In stage 1 (Node), React is built to static production files inside a /dist folder. In stage 2 (Python), these files are copied into the Python environment. FastAPI then mounts /dist to serve frontend assets, running both backend and frontend on a single unified port (9000).

4. Dockerignore Optimizations:
Without a proper .dockerignore file, running 'COPY .' copies massive model files (if previously downloaded) into the build context, crashing Docker with disk exhaustion errors. Adding 'backend/models/' and 'backend/venv/' to .dockerignore prevents this, keeping build context under 5MB.

5. Multi-Architecture Builds (Docker Buildx):
To distribute the image to both Windows users (amd64) and Mac users (arm64), we compile the image utilizing Docker Buildx, pushing a unified multi-arch manifest tag (prathammk01/vocalnote) to Docker Hub.